

CSE2008 – Operating Systems

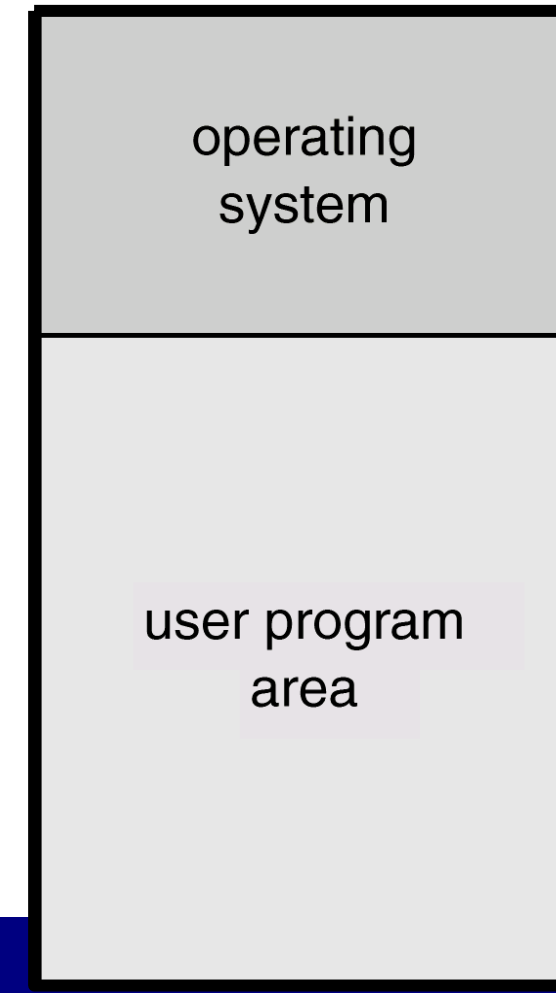
Dr. Venkata Rami Reddy Ch
Sr.Assistant Professor
School of Computer Science & Engineering
VIT-AP University

- static and dynamic memory, memory allocation to a process, continuous memory allocation, non-contiguous memory allocation, paging, segmentation, paging with segmentation.

- A main memory is an **array of addressable** words.
- Memory management is the process of
 - Allocating main memory to user programs
 - Deallocating that memory when it is no longer needed
 - Protecting each user's memory area from other user programs

Main memory usually into two **partitions**:

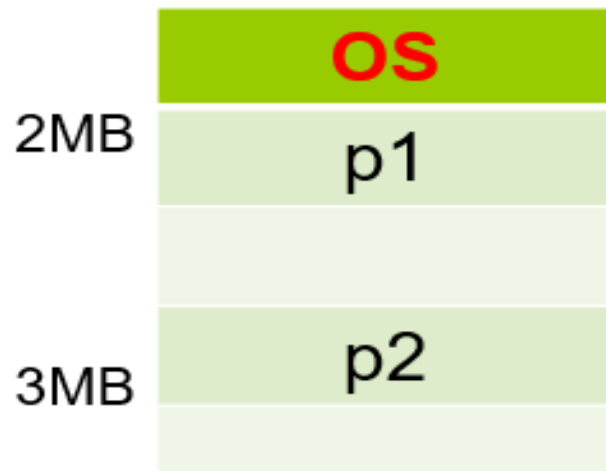
- Resident operating system, usually held in high memory
- User processes then held in low memory



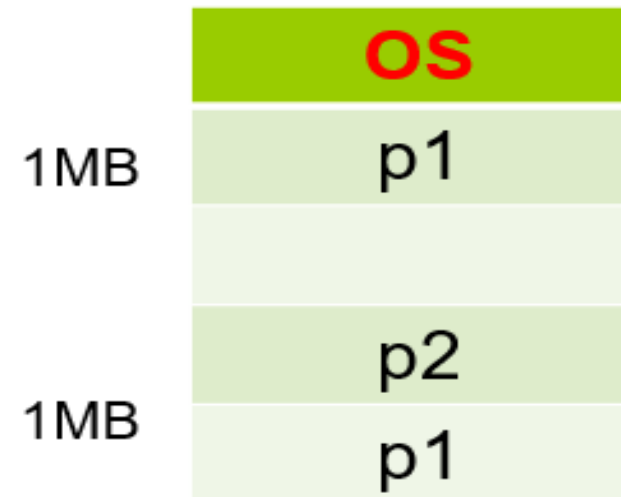
1. Contiguous Memory Allocation
2. Segmentation
3. Paging

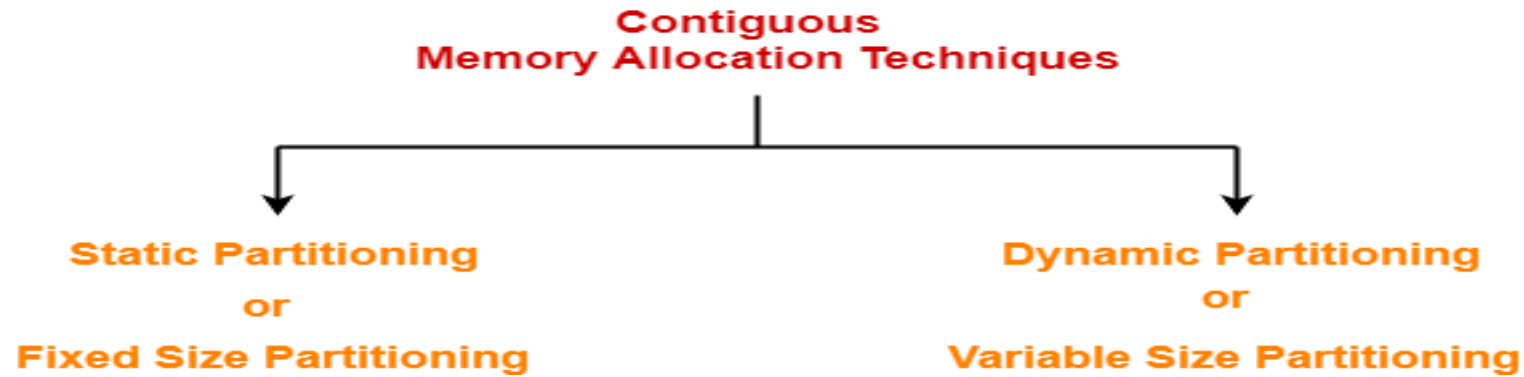
- Each process is allocated a **single contiguous block** of memory
- In contiguous memory allocation, each process contained in a **single contiguous section** of memory

□ Contiguous



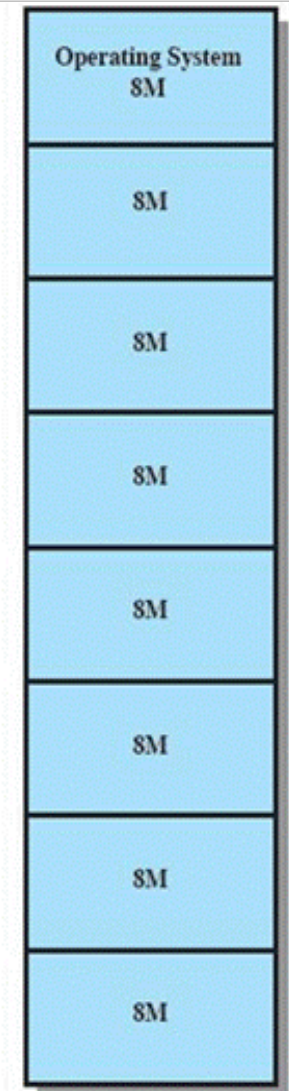
non- Contiguous





Fixed Partitioning

- Memory is divided to **fixed-sized** partitions
- Each partition holds one process.
- Any process whose size is less than or equal to the partition size can be loaded into an available partition



(a) Equal-size partitions

Dynamic Partitioning

- Partitions are created dynamically based on process size
- Memory is divided into dynamically so that **size of each partition** equals to the **size of loaded process**.
- Process is allocated exactly as much memory as it requires
- **Hole** – block of **available** memory;
- holes of various sizes are **scattered** throughout memory

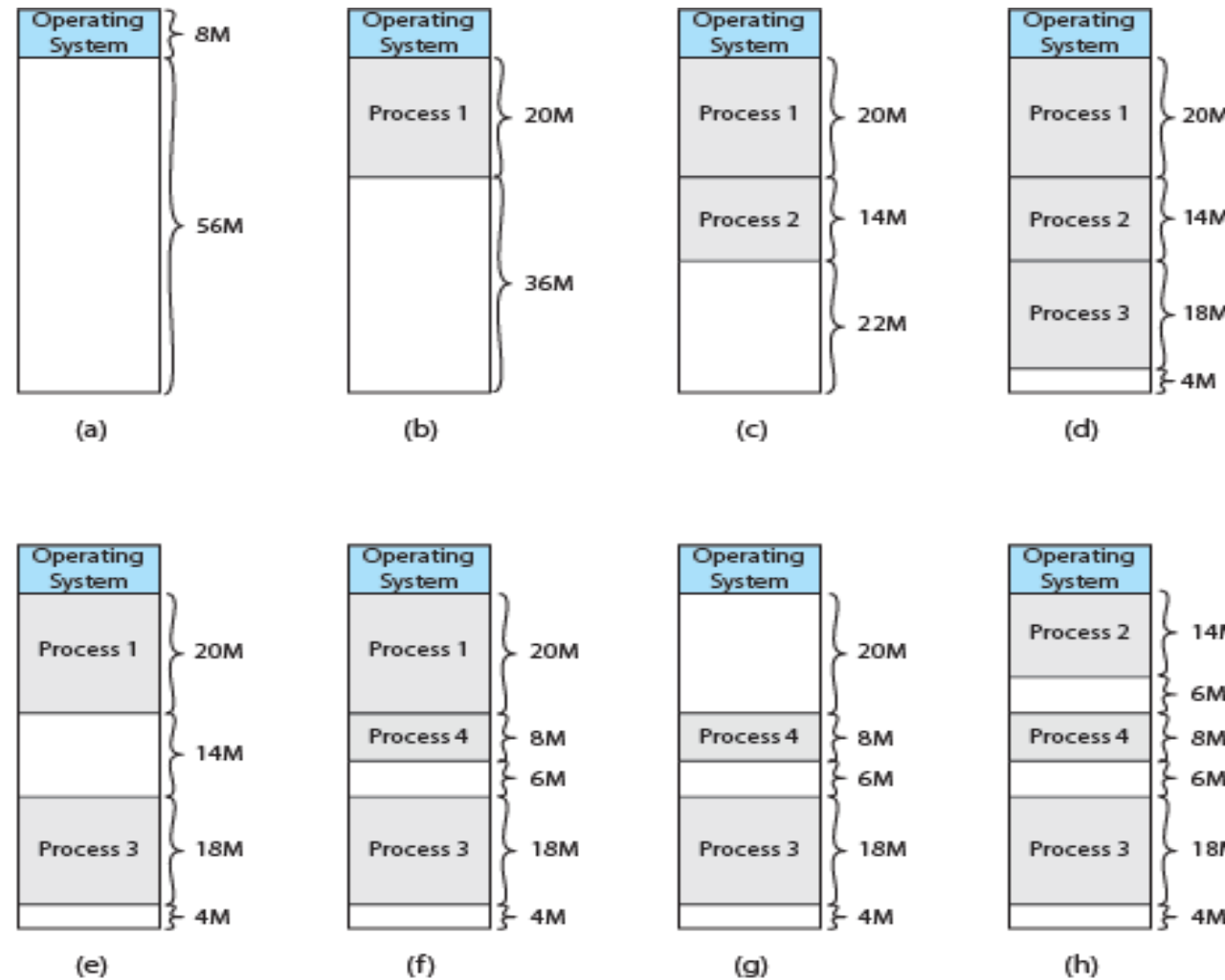
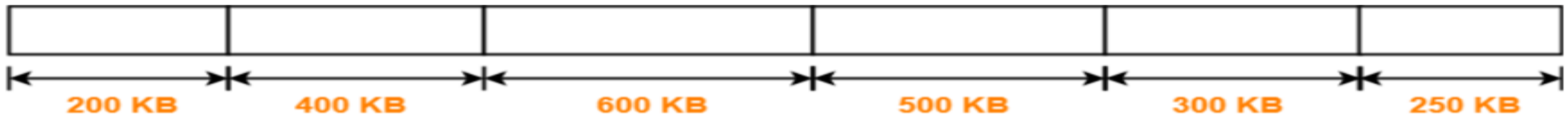


Figure 7.4 The Effect of Dynamic Partitioning

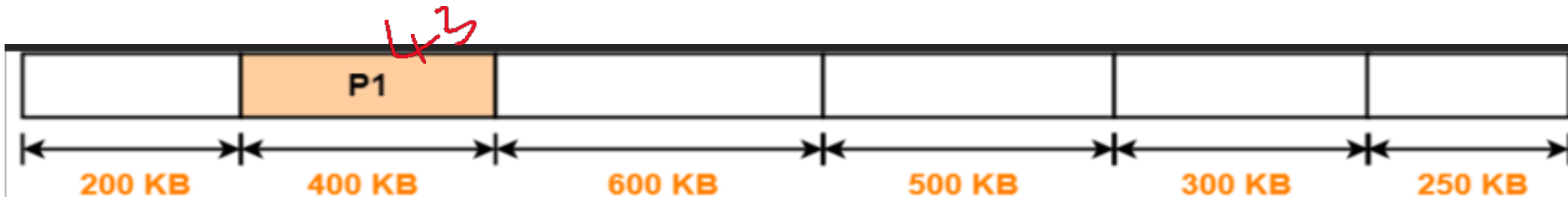
1. First-fit
2. Best-fit
3. Worst-fit

First-fit

- Allocate the *first* hole that is *big enough*
- Begins to scan memory from the beginning and chooses the *first* available *block* that is *large enough*
- We can stop searching as soon as we find a free hole that is large enough.
- Ex: Process P1=357 KB



Memory allocation for Process P1 using First-fit

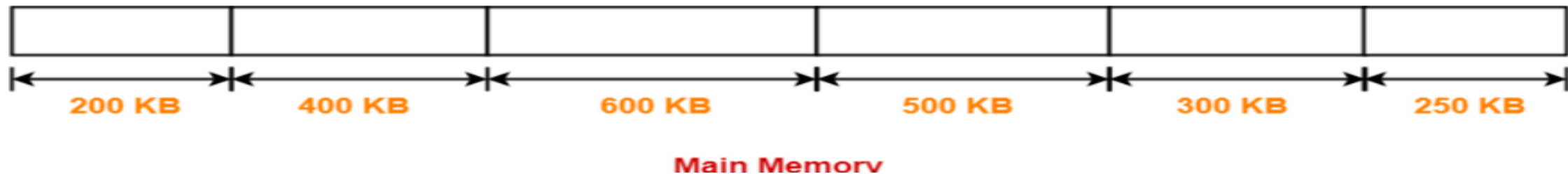


Best-fit

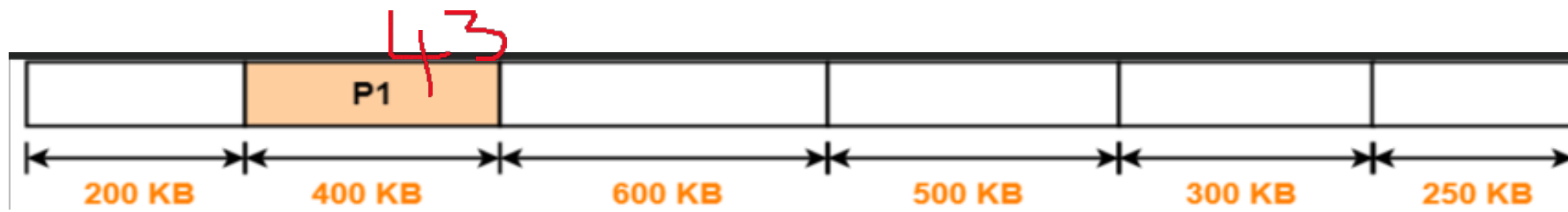


- Allocate the *smallest* hole that is *big enough*;
- We must search *entire list*, unless ordered by size
- This strategy produces the *smallest leftover* hole
- chooses the *block* that is *closest in size* to the request

Ex: Process P1=357 KB



Memory allocation for Process P1 using Best-fit

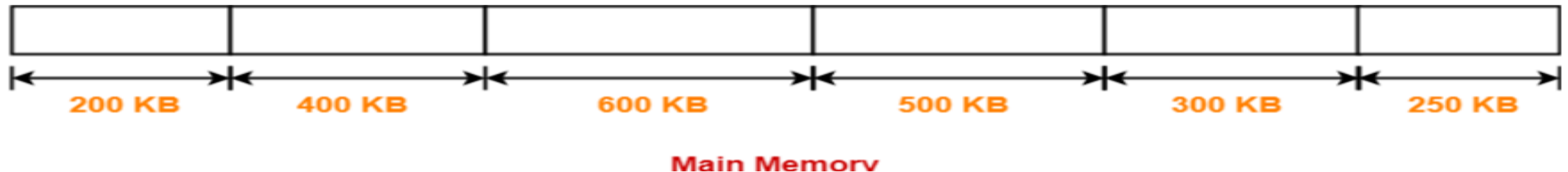


Worst-fit

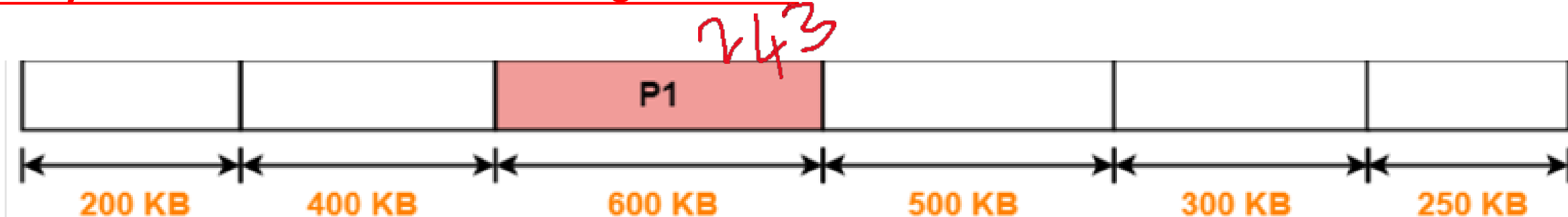


- Allocate the **largest** hole;
- We must also search the **entire list**
- This strategy produces the **largest leftover** hole
- First-fit and best-fit better than worst-fit in terms of **speed and storage utilization**

Ex: Process **P1=357 KB**



Memory allocation for Process P1 using Worst-fit



Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.

Perform the allocation of processes using-

1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

The main memory has been divided into fixed size partitions as-

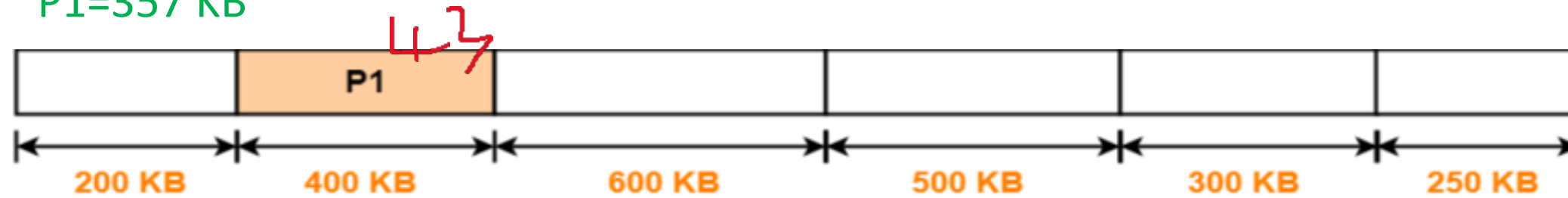


Let us say the given processes are-

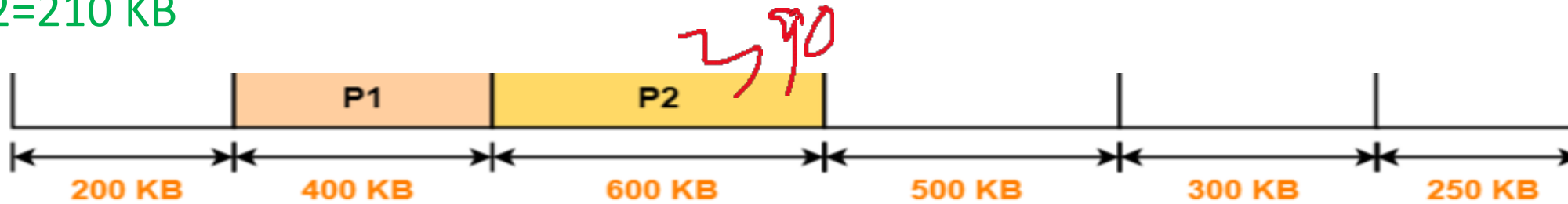
- Process P1 = 357 KB
- Process P2 = 210 KB
- Process P3 = 468 KB
- Process P4 = 491 KB

Allocation using First-fit

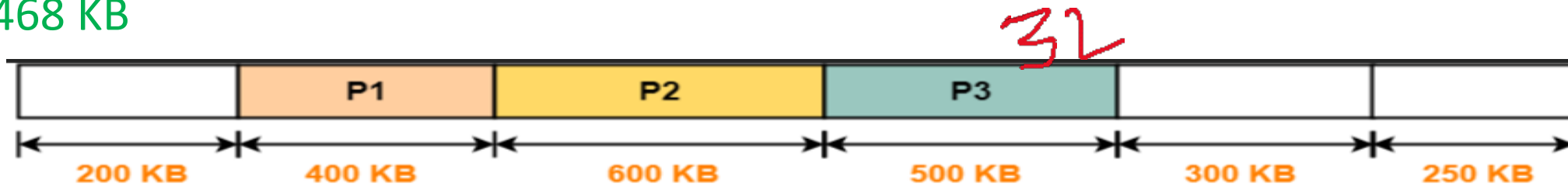
P1=357 KB



P2=210 KB



P3=468 KB

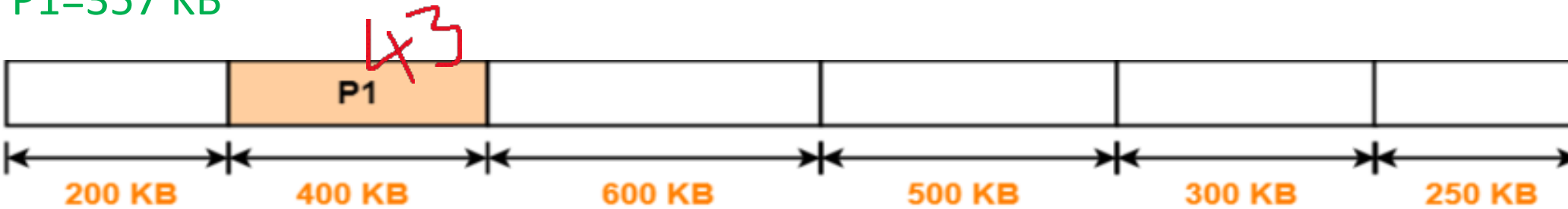


P4=491 KB

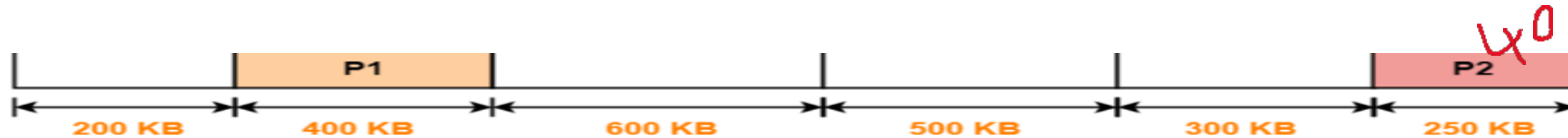
- Process P4 can not be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P4 is available.

Allocation using Best-fit

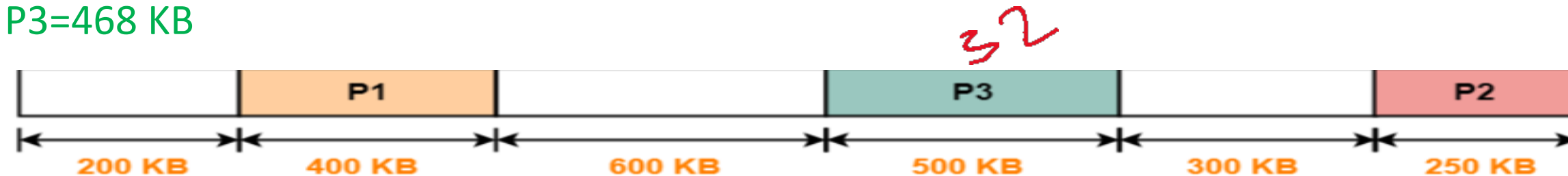
P1=357 KB



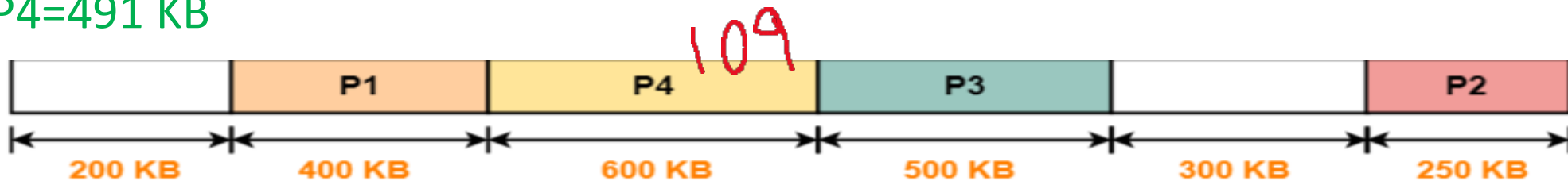
P2=210 KB



P3=468 KB

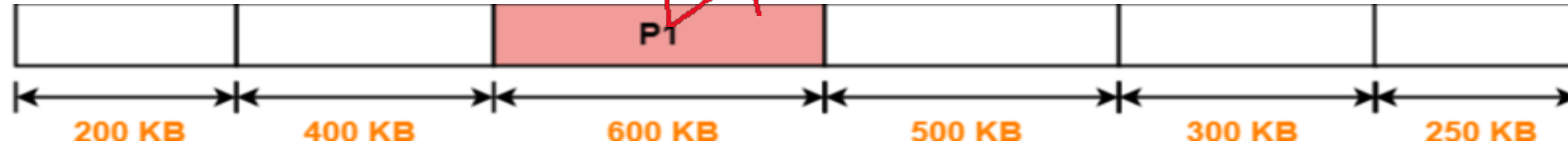


P4=491 KB

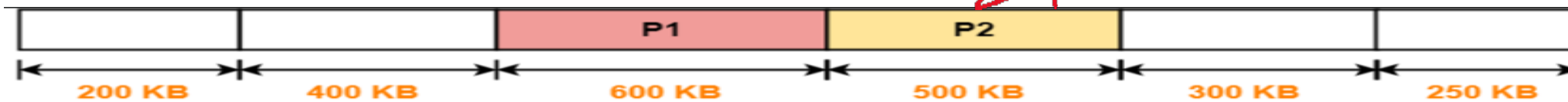


Allocation using Worst-fit

P1=357 KB



P2=210 KB



P3=468 KB

P4=491 KB

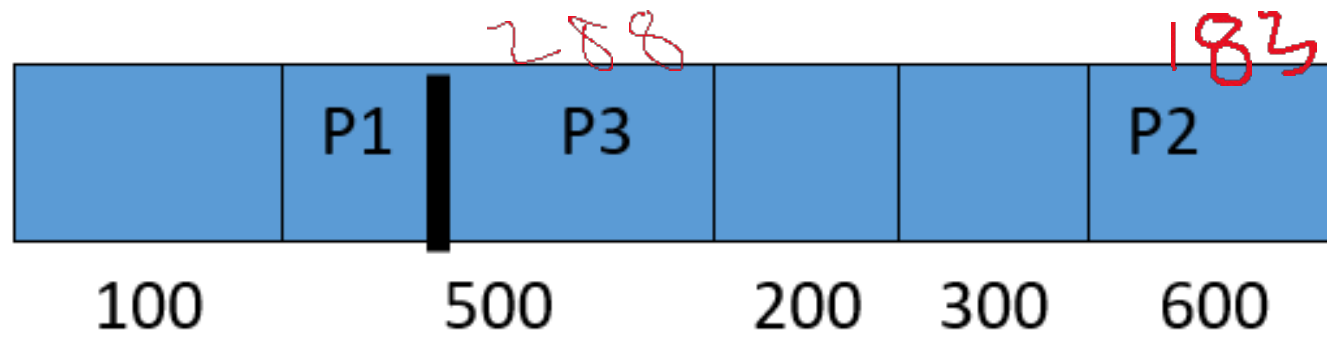
- Process P3 and Process P4 can not be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P3 and process P4 is available.

Problem-2

Given five memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB (in order), how would each of the first-fit, best-fit, and worst-fit algorithms place processes of 212 KB, 417 KB, 112 KB, and 426 KB (in order)? Which algorithm makes the most efficient use of memory?

a. First-fit:

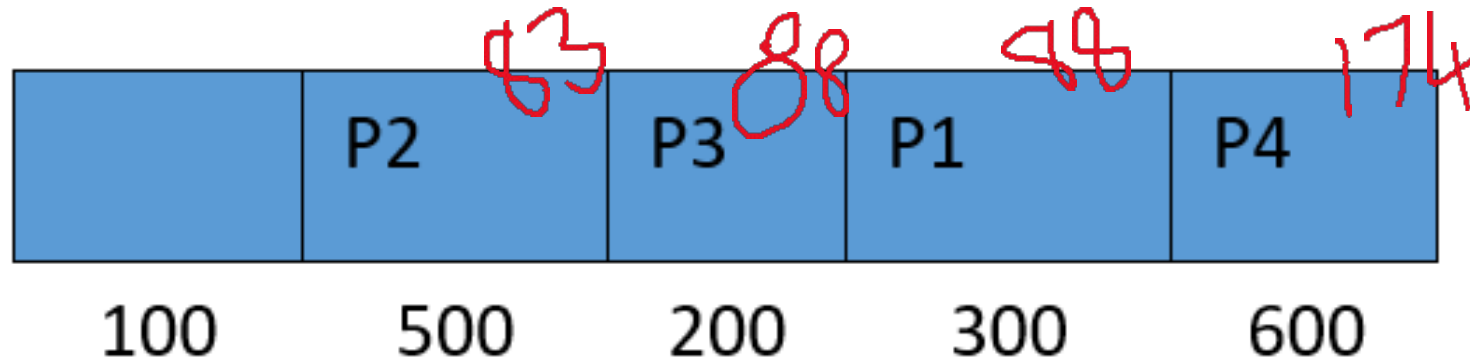
P1=212 KB, P2=417 KB, P3=112 KB, P4=426 KB



1. 212K is put in 500K partition
2. 417K is put in 600K partition
3. 112K is put in 288K partition (new partition 288K = 500K – 212K)
4. 426K must wait

b. Best-fit:

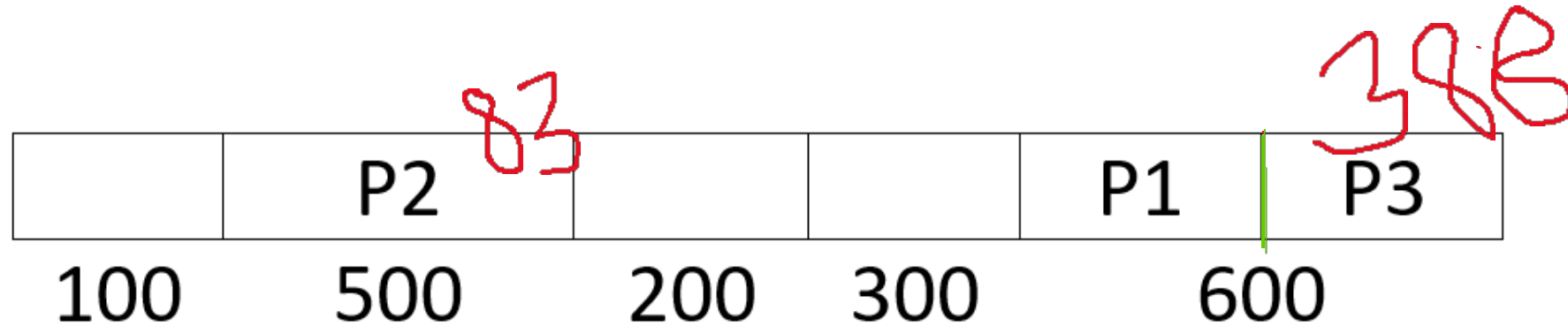
P1=212 KB, P2=417 KB, P3=112 KB, P4=426 KB



1. 212K is put in 300K partition
2. 417K is put in 500K partition
3. 112K is put in 200K partition
4. 426K is put in 600K partition

c. Worst-fit:

P1=212 KB, P2=417 KB, P3=112 KB, P4=426 KB



1. 212K is put in 600K partition
2. 417K is put in 500K partition
3. 112K is put in 388K partition
4. 426K must wait

In this example, best-fit turns out to be the best.

Fragmentation

- It is a phenomenon in which storage space is used inefficiently.

Internal Fragmentation

- It is a phenomenon in which there is wasted space internal to partition due to size of loaded process is smaller than the partition size.
- It occurs in **fixed partitioning**

External Fragmentation

- It exists when there is a total memory space exists to satisfy a request, but available space is not contiguous
- It occurs in dynamic partitioning and segmentation

Example:

Suppose memory has holes: **100 KB, 300 KB, 200 KB**

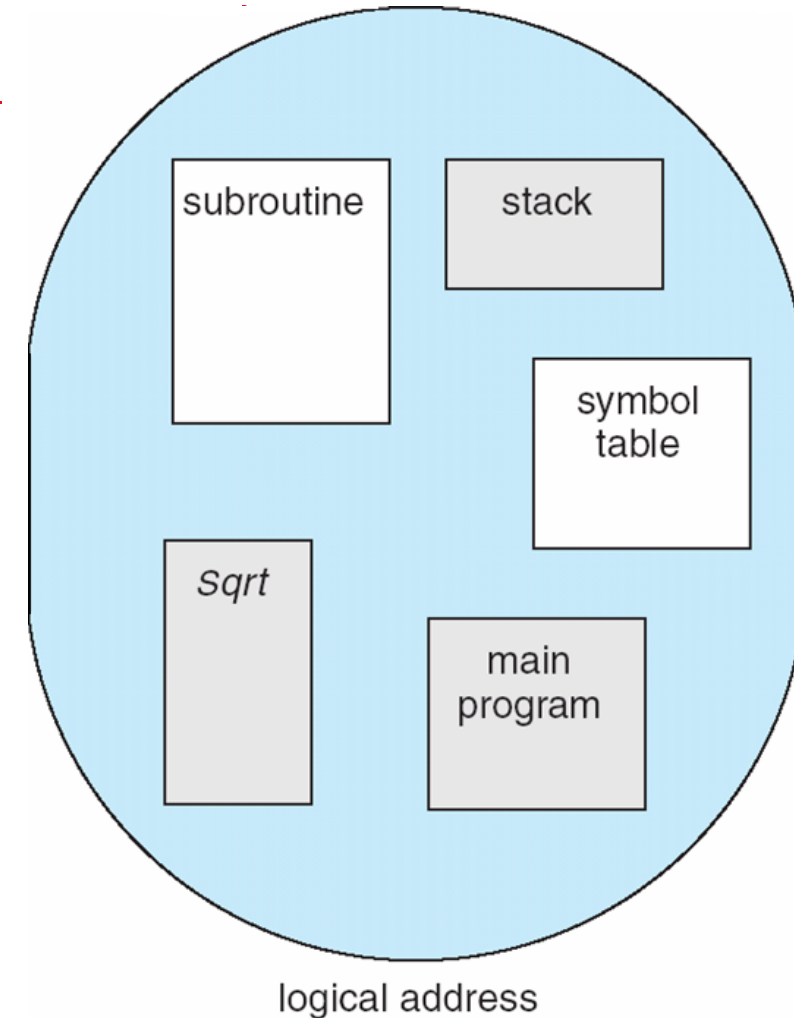
Process requests= **350 KB**

- Can't be allocated even though total free memory = 600 KB → **external fragmentation.**

Segmentation

- **Segmentation** is a **memory management technique** where a **program** is divided into variable-sized **logical segments**.
- A program is a collection of segments
- A segment is a logical unit such as:
 - main program
 - method
 - object
 - local variables, global variables
 - stack
 - symbol table
 - arrays

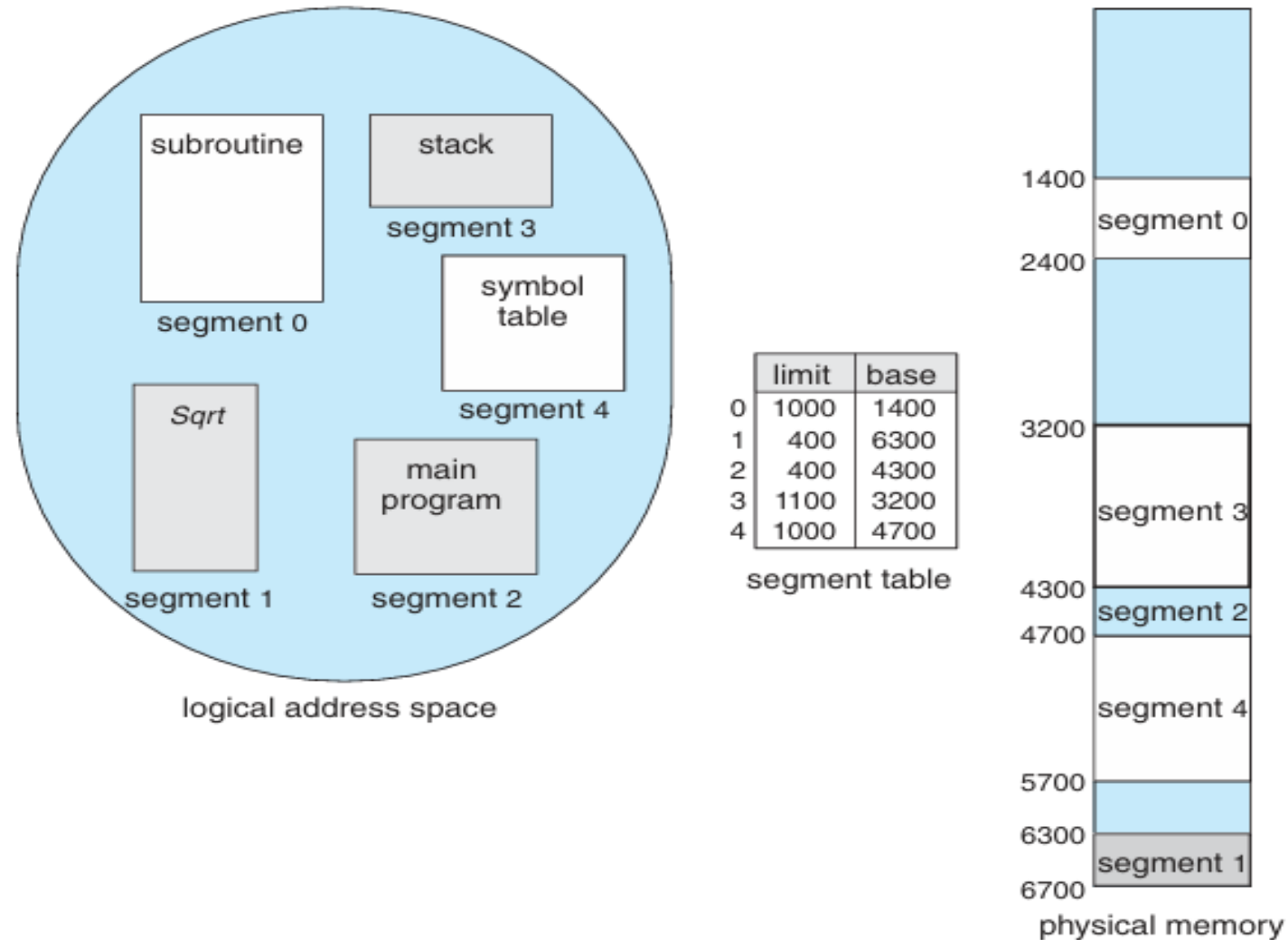
User's View of a Program



Segmentation

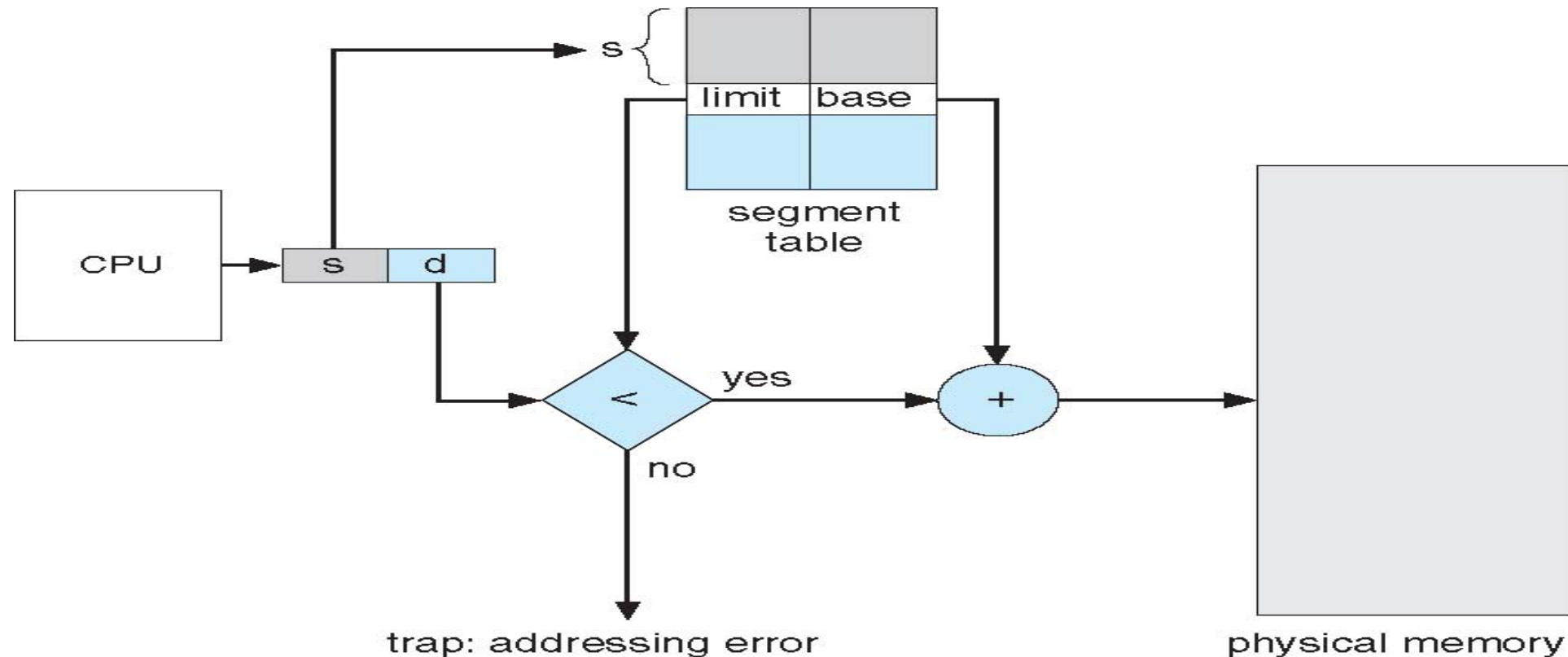
- When the user program is compiled, and the compiler automatically constructs segments reflecting the input program.
- A C compiler might create separate segments for the following:
 1. The code
 2. Global variables
 3. The heap, from which memory is allocated
 4. The stacks used, by each thread
 5. The standard C library

Memory allocation in segmentation.



Segmentation Hardware

- Segmentation hardware is the part of the computer's **memory management unit (MMU)** that supports **segmentation**.



Segmentation Hardware

Components of Segmentation Hardware:

Segment Table

- Each process has a segment table.
- Stores information for each segment:
 - Base Address** → starting physical address of the segment in memory.
 - Limit (Length)** → size of the segment.
- Logical address consists of a two tuple:
 <segment-number, offset>,
- The **segment number** is used as an index to the segment table.
- The **offset** *d must be* less than the segment limit.
- When an **offset** is **legal**, it is **added** to the **segment base** to produce the **address** in **physical memory** of the **desired byte**.

Address Translation in Segmentation

Logical Address = (**Segment number (s)** , **Offset (d)**)

Steps:

1. CPU generates (s, d).
2. Access segment table entry for s:
 1. Get **Base Address** and **Limit**.
3. If $d \geq \text{Limit}$ → **Trap: segmentation fault**.
4. Else,
Physical Address = Base Address + d.

Example

- Segment Table:
 - Seg 0 → Base = 2000, Limit = 600
 - Seg 1 → Base = 4000, Limit = 1000

Logical Address = (1, 500)

- Check: $500 < 1000$ ✓
- Physical Address = $4000 + 500 = \mathbf{4500}$

Problems on segmentation

A system uses segmentation for memory management. The segment table is given below:

Segment No.	Base Address	Length
0	4000	300
1	6000	800
2	9000	1000

Given the following logical addresses in the form (segment number, offset), compute the corresponding physical addresses. If the offset is greater than or equal to the segment length, report a Segment Fault.

Logical Addresses:

1. (0, 250)
2. (1, 700)
3. (2, 1200)
4. (0, 350)

- **Logical Address** = (Segment No., Offset)
- **Physical Address** = Base Address of that segment + Offset
- **Condition:** If offset $\geq$ length, it causes a **Segment Fault**.

Logical Addresses

1. (0, 250)

Segment 0 $\rightarrow$ Base = 4000, Length = 300

Offset = 250 < 300 ✓ valid

Physical Address = 4000 + 250 = **4250**

2. (1, 700)

Segment 1 $\rightarrow$ Base = 6000, Length = 800

Offset = 700 < 800 ✓ valid

Physical Address = 6000 + 700 = **6700**

3. (2, 1200)

• Segment 2 $\rightarrow$ Base = 9000, Length = 1000

• Offset = 1200 < 1000 ✗ invalid

• Result = **Segment Fault**

Segment Table

Segment No.	Base Address	Length
0	4000	300
1	6000	800
2	9000	1000

4. (0, 350)

• Segment 0 $\rightarrow$ Base = 4000, Length = 300

• Offset = 350 < 300 ✗ invalid

• Result = **Segment Fault**

✓ Final Answers

1. (0, 250) $\rightarrow$ **4250**

2. (1, 700) $\rightarrow$ **6700**

3. (2, 1200) $\rightarrow$ **Segment Fault**

4. (0, 350) $\rightarrow$ **Segment Fault**

Drawbacks of Segmentation

1. External Fragmentation
2. Complex Memory Allocation
3. Unequal Segment Sizes
4. Overhead of Segment Table
5. Protection and Sharing Complexity
6. Difficulty in Compaction

External Fragmentation in segmentation.

- Segmentation suffers with External Fragmentation
- When processes are loaded and removed, free memory gets divided into **scattered holes**.
- A new segment might be **larger than any single hole**, even if the **total free memory is enough**

Why Segmentation Suffers External Fragmentation?

- Segments are **not fixed size** (variable).
- Each segment needs a **continuous block of memory**.
- As processes come and go, **free blocks are scattered**, leaving gaps.

Example:

Suppose memory has holes: **100 KB, 300 KB, 200 KB**

Process requests segment = **350 KB**

Can't be allocated even though total free memory = 600 KB → **external fragmentation**.

Paging

- Paging is a **memory management technique** used in operating systems to efficiently manage process memory.
- It avoids external fragmentation by dividing both physical memory and logical memory(process) into fixed-size blocks.

Frames:

- Physical memory is divided into fixed-sized blocks called **frames**.
- Size is power of 2, between 512 bytes and 16 Mbytes

Pages:

- Logical memory(process) is divided into blocks of same size called **pages**.
- To run a program of size **N** pages, need to find **N** free frames and load program

Paging Model of Logical and Physical Memory

page 0
page 1
page 2
page 3

logical
memory

0	1
1	4
2	3
3	7

page table

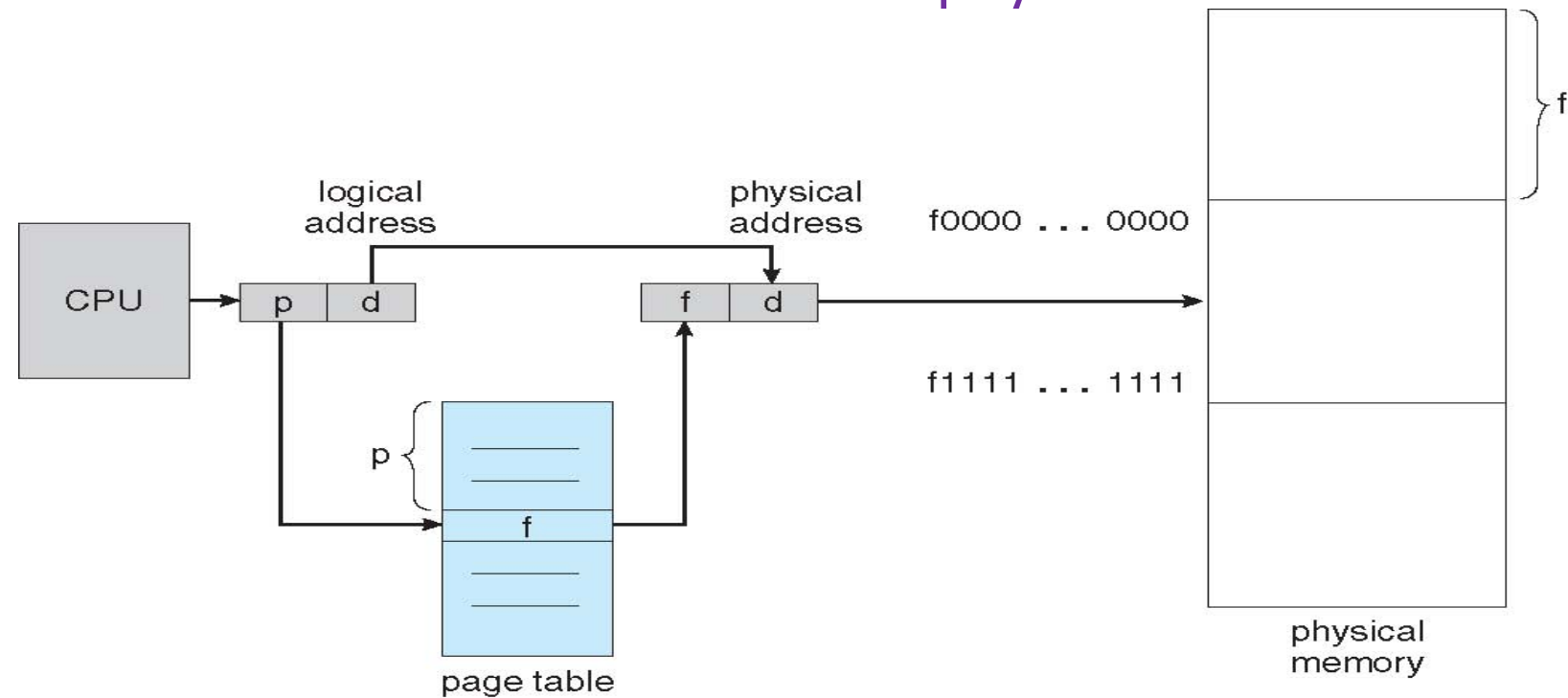
frame
number

0	
1	page 0
2	
3	page 2
4	page 1
5	
6	
7	page 3

physical
memory

Address Translation Scheme or hardware support

- The hardware support for paging is illustrated in Figure.
- Every address generated by the CPU is divided into two parts :a **page number(p)** and a **page offset(d)**.
- The **page number (p)** is used as an index into a **page table**.
- The **page table** contains **frame number** of each page in **physical memory**
- **page offset(d)** is combined with **frame number** to define the **physical address** that is sent to the memory unit



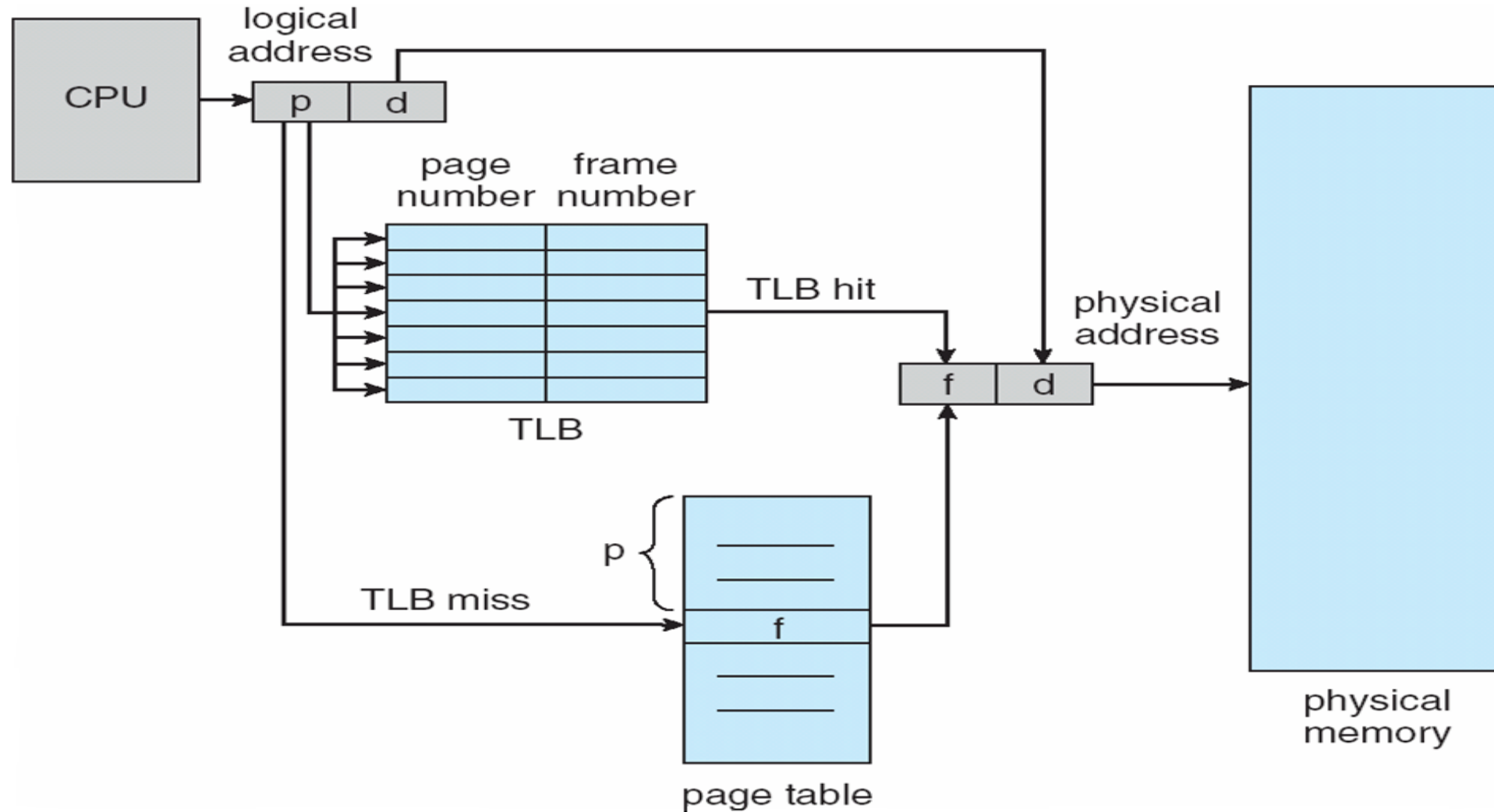
Implementation of Page Table

- The hardware implementation of the page table can be done in several ways.
- In one approach, the page table is kept in **main memory** and a **Page-table base register (PTBR)** points to the page table
- The problem with this approach is the time required to access a user memory location.
- With this scheme, two memory accesses are needed to access a byte
 - one to access page table entry in the memory
 - one to access a byte from the memory
- Thus, memory access is slowed by a factor of 2
- This delay would be intolerable under most circumstances.

Translation look-aside buffers (TLBs)

- The standard solution to this problem is to use a **special, small, fast lookup hardware** cache called a **translation look-aside buffer (TLB)**.
- Each entry in the TLB consists of two parts: **page number** and **frame number**
- The TLB contains only a **few** of the **page-table entries**.
- When a logical address is generated by the CPU, its page number is presented to the TLB.
- If the page number is **found in TLB**, its frame number is used to access memory.
- If the page number is **not in the TLB** (known as a **TLB miss**), a memory reference to the page table must be made.
- When the frame number is obtained from page table , we can use it to access memory.
- In addition, we add the page number and frame number to the TLB so that they will be found quickly on the next reference.
- If the TLB is already full of entries, an existing entry must be selected for replacement

Paging Hardware With TLB



Paging Hardware With TLB

- The percentage of times that the page number is found in the TLB is called the **hit ratio**.
- An 80-percent hit ratio, for example, means that we find the desired page number in the TLB 80 percent of the time.
- If it takes 20ns to search the TLB and 100ns to access memory, then a mapped-memory access takes 120 nanoseconds when the page number is in the TLB.
- If we fail to find the page number in the TLB(20ns) then we must first access memory for the page table and frame number (100 ns) and then access the desired byte in memory(100 ns), for a total of 220 nanoseconds.

$$\text{effective access time} = 0.80 \times 120 + 0.20 \times 220$$

$$= 140 \text{ nanoseconds}$$

Effective Access Time (EAT)

- Let:
 - t_{TLB} = Time to search TLB
 - t_M = Time to access main memory
 - h = Hit ratio

Case 1: TLB Hit

- Access TLB + 1 memory access

$$= t_{TLB} + t_M$$

Case 2: TLB Miss

- Access TLB + 1 memory access (to get page table entry) + 1 memory access (to get data)

$$= t_{TLB} + 2t_M$$

Formula:

$$EAT = h \times (t_{TLB} + t_M) + (1 - h) \times (t_{TLB} + 2t_M)$$

A computer system uses paging with a **TLB access time of 20 ns** and **main memory access time of 100 ns**. 1. Calculate the EAT for hit ratios of **80%, 95%, and 99%**. 2. What is the **best-case** and **worst-case** access time?

◆ Formula for Effective Access Time (EAT)

$$EAT = h \times (t_{TLB} + t_M) + (1 - h) \times (t_{TLB} + 2t_M)$$

Where:

- $t_{TLB} = 20 \text{ ns}$
- $t_M = 100 \text{ ns}$
- $h = \text{hit ratio}$

Case 1: $h = 80 \% = 0.8$

$$EAT = 0.8(20 + 100) + 0.2(20 + 200)$$

$$EAT = 0.8(120) + 0.2(220)$$

$$EAT = 96 + 44 = 140 \text{ ns}$$

Case 2: $h = 95 \% = 0.95$

$$EAT = 0.95(120) + 0.05(220)$$

$$EAT = 114 + 11 = 125 \text{ ns}$$

Case 3: $h = 99 \% = 0.99$

$$EAT = 0.99(120) + 0.01(220)$$

$$EAT = 118.8 + 2.2 = 121 \text{ ns}$$

• **Best Case (100% hit ratio):**

$$EAT = 20 + 100 = 120 \text{ ns}$$

• **Worst Case (0% hit ratio):**

$$EAT = 20 + 200 = 220 \text{ ns}$$

pros and cons of paging

Pros

- No external fragmentation
- efficient allocation.

Cons

- Internal fragmentation,
- page table overhead,
- slower without TLB.

Where does Internal fragmentation occur?

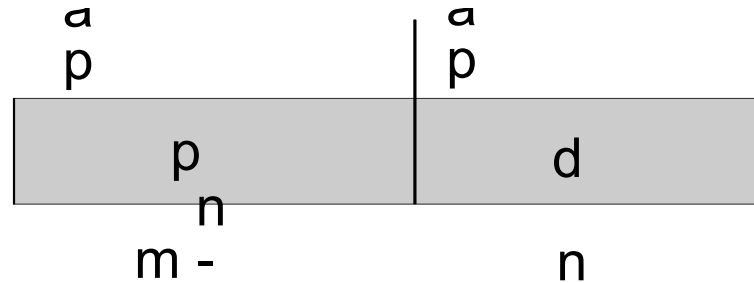
- The last page of a process **may not completely fill its allocated frame.**

Example:

- Page size = 1 KB, Process size = 4.2 KB
- Process requires 5 pages → 5 frames allocated.
- First 4 pages (4 KB) fit exactly.
- Last page needs only 0.2 KB, but the whole 1 KB frame is allocated.
- **Wasted = 0.8 KB (internal fragmentation)**

Page Table Entries

- For given logical address space 2^m and page size 2^n



- $m-n$ bits are used to represent page number and n bits are used to represent page offset.

$$\text{Number of Page Table Entries (PTE)} = \frac{\text{Logical Address Space Size}}{\text{Page Size}}$$

$$\text{Number of Entries} = \frac{2^m}{2^n} = 2^{(m-n)}$$

A computer system uses a 32-bit logical address space and a page size of 4 KB.

(a) How many entries are required in the page table?

(b) If each page table entry requires 4 bytes, what is the total size of the page table?

(a) Number of entries in the page table

- Logical address = **32 bits** (2^{32} bytes address space).

- Page size = **4 KB** = 2^{12} . ($4 \times 1024 = 2^2 \times 2^{10}$)

- No. of page table entries = $2^{32} / 2^{12} = 2^{32-12} = 2^{20} = 1,048,576$.

So, the page table has about **1 million entries**.

(b) Total page table size

$$\begin{aligned} \text{Page Table Size} &= \text{No. of entries} \times \text{Size of each entry} \\ &= 1,048,576 \times 4 \text{ bytes} = 4,194,304 \text{ bytes} \\ &= 4 \text{ MB} \end{aligned}$$

A computer system has a **16-bit logical address space** and uses a **page size of 4 KB**. The physical memory size is **64 KB**.

- i. Calculate the **number of pages** in the logical address space.
- ii. Determine the **number of bits required to specify the page number**.
- iii. Determine the **number of bits required for the page offset**.
- iv. Calculate the **number of frames** in physical memory.

Given:

- Logical address = 16 bits $\rightarrow 2^{16}$
- Page size = 4 KB = 2^{12}
- Physical memory = 64 KB

1 Number of pages in logical address space

$$\text{No. of pages} = \frac{\text{Logical Address Space}}{\text{Page Size}} = \frac{2^{16}}{2^{12}} = 2^4 = 16$$

✓ **Answer:** 16 pages

2 Number of bits to specify the page number

$$\text{No of bits for Page number} = m - n = 16 - 12 = 4 \text{ bits}$$

✓ **Answer:** 4 bits

3 Number of bits for page offset

$$\text{No of bits for Page offset} = n \text{ bits} = 12 \text{ bits}$$

✓ **Answer:** 12 bits

4 Number of frames in physical memory

$$\begin{aligned} \text{No. of frames} &= \frac{\text{Physical Memory Size}}{\text{Frame Size}} = \frac{64 \text{ KB}}{4 \text{ KB}} \\ &= 16 \end{aligned}$$

✓ **Answer:** 16 frames

Address translation in paging

Physical address = Frame Number $\times$ Page Size(bytes) + Offset.

1) A computer system uses **paging** with a page size of **1 KB**. A process generates a logical address that corresponds to **page number 5** with an **offset of 300 bytes**. The page table indicates that **page 5 is stored in frame 2**.

Physical Address = $2 \times 1024 + 300 = 2348$

2) logical address given as Decimal number

Given, **page size = 1 KB**

logical address (decimal) = **2348**.

Page table: **page 2 $\rightarrow$ frame 5**.

Steps

1. Page number = $\lfloor 2348 / 1024 \rfloor = 2$

2. Offset = $2348 \bmod 1024 = 300$

3. Physical = frame $\times$ page_size + offset = $5 \times 1024 + 300 = \mathbf{5420 \text{ (decimal)}}$

Address translation in paging

Binary logical address

Given ,**page size = 2 KB = $2 \times 1024 = 2^{11}$ bytes**(offset = 11 bits)

logical address (binary) = **01010 11001010101₂**

(here left 5 bits = page, right 11 bits = offset)

So page = $01010_2 = 10$, offset = $11001010101_2 = 1621$.

Page table: **page 10 → frame 3.**

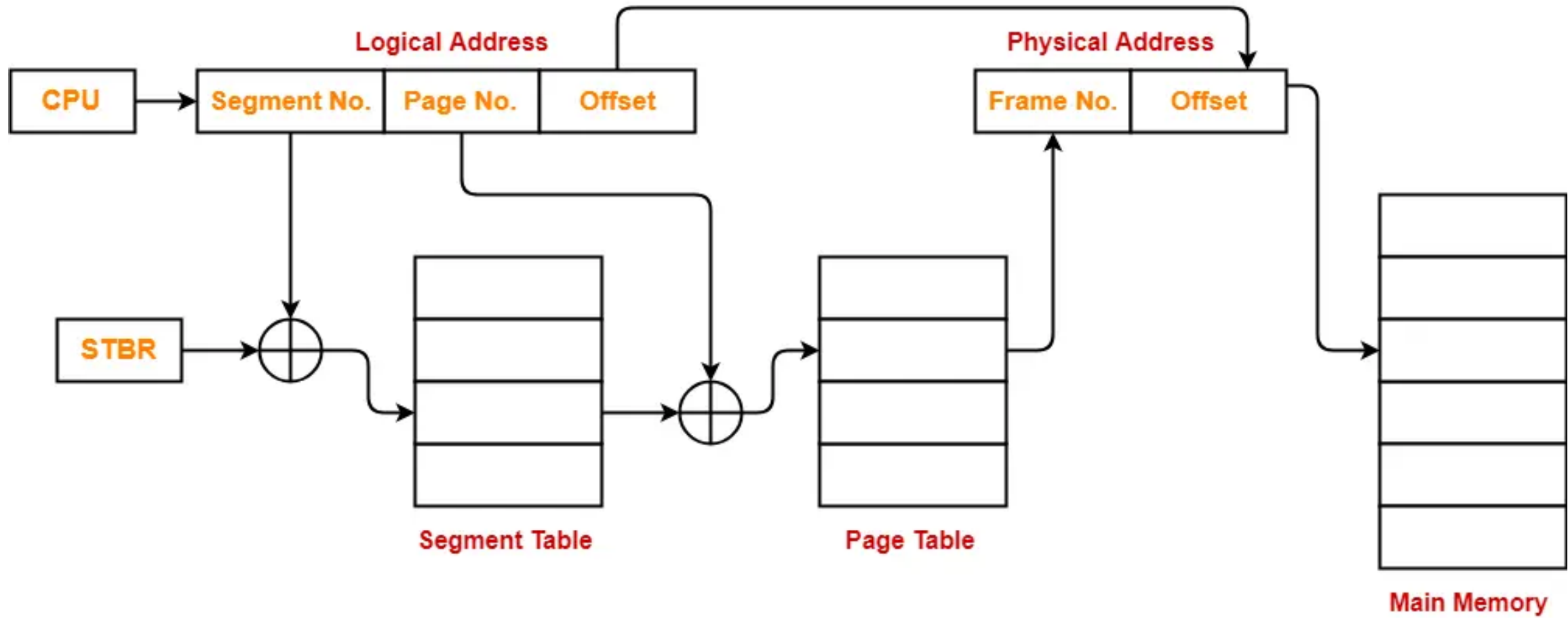
Physical = $3 \times 2048 + 1621 = 6144 + 1621 = \mathbf{7765 \text{ (decimal)}}$

physical address in binary: **0001 1110 0100 0101₂**

paging with segmentation

- Paging with segmentation is a **hybrid memory management technique** that combines **segmentation** and **paging**.
- **Process** is first divided into **segments** and then **each segment** is divided into **pages**.
- These **pages** are then **stored** in the **frames** of main memory.
- A **page table** exists for **each segment** that keeps track of the **frames** storing the **pages** of that segment.
- Each page table occupies one frame in the main memory.
- A **segment table** exists for **each process** that keeps track of the **frames** storing the **page tables** of segments.

Translating Logical Address into Physical Address



Translating Logical Address into Physical Address

Translating Logical Address into Physical Address

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of three parts-

< Segment Number, Page Number, Offset >

- **Segment Number (s)**: Identifies which segment.
- **Page Number (p)**: Identifies the page within that segment.
- **Offset (d)**: Displacement within the page.

Step-02: Use **Segment Number** to index into the **Segment Table** → get the **Frame number** of the frame which contains the **Page Table** for that segment.

Step-03: Use **Page Number** to index into that **Page Table** → get the **Frame Number** of the frame which contains the **Page**

Step-04:

Physical Address = Frame Number + Offset (d)

Translating Logical Address into Physical Address

In **paging with segmentation**, a logical address usually has **three fields**:

(segment number, page number, page offset)

Number of bits for segment number

• Suppose the **maximum number of segments** is $\mathcal{S}$.

$$\text{bits for segment} = \lceil \log_2 \mathcal{S} \rceil$$

Example: If there are 8 segments $\rightarrow \lceil \log_2 8 \rceil = 3\text{bits}$

Number of bits for page number

• Suppose **segment i** has maximum P_i pages.

$$\text{bits for page} = \lceil \log_2 P_i \rceil$$

Example: If a segment has 16 pages $\rightarrow \lceil \log_2 16 \rceil = 4\text{bits}$

Number of bits for page offset

• Suppose **page size** = B bytes.

$$\text{bits for offset} = \lceil \log_2 B \rceil$$

Example: Page size = 1024 bytes $\rightarrow \lceil \log_2 1024 \rceil = 10\text{bits}$

Total logical address bits = *segment bits* + *page bits* + *offset bits*

A system has the following:

- Page size = 1 KB (1024 bytes)

Segment table:

Segment #	Page Table Base Address	Limit (#pages)
0	1000	4
1	1050	3

Page tables:

Segment 0 Page Table

Page #	Frame #
0	5
1	9
2	12

Translate the logical address
(segment=0, page=2, offset=300)
into **physical address**.

Step 1: Identify the Segment

- Logical address segment = 0
 - Segment table entry → base address of segment 0 page table = 1000
-

Step 2: Identify the Page

- Logical address page = 2
 - Look up **segment 0's page table** at page 2 → frame number = 12
-

Step 3: Physical Address Calculation

Formula:

$$\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$$

Substitute values:

$$\text{Physical Address} = (12 \times 1024) + 300$$

Step by step:

1. $12 \times 1024 = 12288$
2. Add offset: $12288 + 300 = 12588$

A computer system uses **paging with segmentation**. The system has the following configuration:

Segment Table:

Segment Number	Page Table Base Address	Segment Limit (Number of Pages)
0	1000	4
1	2000	5
2	3000	3
3	4000	2

Page Table for Segment 1:

Page Number	Frame Number
0	4
1	7
2	2
3	6
4	3

- **Page size:**2 KB
 - **Logical Address (16 bit binary):**01 010 0011010110
1. Identify the **segment number**, **page number**, and **offset** from the given logical address.
 2. Verify whether the segment number is valid (within the segment table limit).
 3. Determine the **frame number** corresponding to the page number in the segment's page table.
 4. Compute the **physical address** using the formula:

Step 1: Extract Segment Number, Page Number, and Offset

Logical address binary:

01 010 0011010110

A **16-bit logical address** is divided into **Segment | Page | Offset**.

We need **S, P, D bits**:

Offset (D bits): Depends on page size

- Page size = 2 KB = 2^{11} bytes $\rightarrow$ Offset needs **11 bits**

Segment Number (S bits): Depends on number of segments

- Suppose 4 segments $\rightarrow$ Segment bits = 2 ($2^2 = 4$)

Page Number (P bits): Remaining bits

$$P = 16 - S - D = 16 - 2 - 11 = 3$$

- **Segment (2 bits)** = 01 $\rightarrow$ Decimal = **1**
- **Page (3 bits)** = 010 $\rightarrow$ Decimal = **2**
- **Offset (11 bits)** = 0011010110 $\rightarrow$ Decimal =

Convert offset 0011010110 to decimal: 430

$$0 * 2^9 + 0 * 2^8 + 1 * 2^7 + 1 * 2^6 + 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

Step 2: Verify Segment Validity

Segment number = 1

Segment 1 in segment table $\rightarrow$ Segment Limit = 5 $\rightarrow$ **valid**

Step 3: Find Frame Number

Segment = 1 $\rightarrow$ Use its page table

Page number = 2 $\rightarrow$ Frame number = **2**

Step 4: Compute Physical Address

Formula:

$$\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$$
$$\text{Physical Address} = (2 \times 2048) + 430 = 4096 + 430 = 4526$$